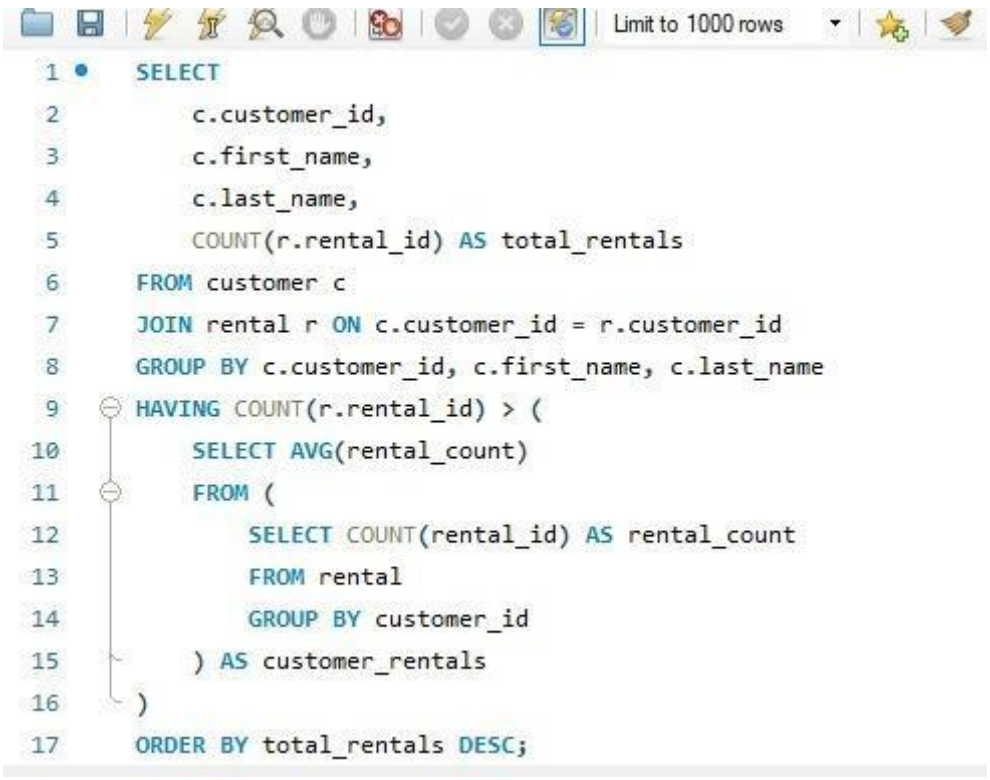
Subqueries, Windows Function, CTE, Views and Stored Procedure Assignment

Q1. Display the names of customers who have rented more movies than the average number of rentals made by all customers.

Sol 1 : ***Query:***



Output:

Result Grid | Filter Rows: | Export:

customer_id	first_name	last_name	total_rentals
292	MISTY	LAMBERT	27
331	ERIC	ROBERT	27
333	ANDREW	PURDY	27
383	MARTIN	BALES	27
396	EARL	SHANKS	27
407	DALE	RATCLIFF	27
421	LEE	HAWKS	27
426	BRADLEY	MOTLEY	27
434	EDDIE	TOMLIN	27
471	DEAN	SAUER	27
491	RICK	MATTOX	27
498	GENE	SANBORN	27
561	IAN	STILL	27
577	CLIFTON	MALCOLM	27
579	DARYL	LARUE	27
580	ROSS	GREY	27
581	VIRGIL	WOFFORD	27
591	KENT	ARSENAULT	27
594	EDUARDO	HIATT	27

Result Grid | Filter Rows: | Export:

customer_id	first_name	last_name	total_rentals
575	ISAAC	OGLESBY	29
588	MARION	OCAMPO	29
592	TERRANCE	ROUSH	29
6	JENNIFER	DAVIS	28
12	NANCY	THOMAS	28
14	BETTY	WHITE	28
16	SANDRA	MARTIN	28
49	JOYCE	EDWARDS	28
57	EVELYN	MORGAN	28
77	JANE	BENNETT	28
90	RUBY	WASHIN...	28
92	TINA	SIMMONS	28
126	ELLEN	SIMPSON	28
132	ESTHER	CRAWFO...	28
145	LUCILLE	HOLMES	28
166	LYNN	PAYNE	28
187	BRITTANY	RILEY	28
213	GINA	WILLIAM...	28
221	BESSIE	MORRISON	28
262	PATSY	DAVIDSON	28

Result 1

Q2. Display each film along with its rental rate and assign a rank based on rental rate from highest to lowest.

Sol 2: ***Query:***

```
19
20 •  SELECT
21        title,
22        rental_rate,
23        RANK() OVER (ORDER BY rental_rate DESC) AS rental_rate_rank
24    FROM film;
```

Output:

Result Grid | Filter Rows: | Export:

	title	rental_rate	rental_rate_rank
▶	ACE GOLDFINGER	4.99	1
	AIRPLANE SIERRA	4.99	1
	AIRPORT POLLOCK	4.99	1
	ALADDIN CALENDAR	4.99	1
	ALI FOREVER	4.99	1
	AMELIE HELLFIGHTERS	4.99	1
	AMERICAN CIRCUS	4.99	1
	ANTHEM LUKE	4.99	1
	APACHE DIVINE	4.99	1
	APOCALYPSE FLAMINGOS	4.99	1
	ATTACKS HATE	4.99	1
	ATTRACTION NEWTON	4.99	1
	AUTUMN CROW	4.99	1
	BABY HALL	4.99	1
	BACKLASH UNDEFEATED	4.99	1
	BEAST HUNCHBACK	4.99	1
	BEAUTY GREASE	4.99	1
	BEHAVIOR RUNAWAY	4.99	1
	BETRAYED REAR	4.99	1
	BIKINI BORROWERS	4.99	1

Result 2 ×

Result Grid | Filter Rows: | Export:

title	rental_rate	rental_rate_rank
WYOMING STORM	4.99	1
YENTL IDAHO	4.99	1
ZORRO ARK	4.99	1
ADAPTATION HOLES	2.99	337
AFFAIR PREJUDICE	2.99	337
AFRICAN EGG	2.99	337
AGENT TRUMAN	2.99	337
ALABAMA DEVIL	2.99	337
ALIEN CENTER	2.99	337
ALLEY EVOLUTION	2.99	337
AMISTAD MIDSUMMER	2.99	337
ANALYZE HOOSIERS	2.99	337

Result 2 ×



Result Grid | Filter Rows: | Export: | W

title	rental_rate	rental_rate_rank
JUMPING WRATH	0.99	660
JUNGLE CLOSER	0.99	660
KANE EXORCIST	0.99	660
KARATE MOON	0.99	660
KICK SAVANNAH	0.99	660
KILL BROTHERHOOD	0.99	660
KWAI HOMEWARD	0.99	660
LADYBUGS ARMAGEDDON	0.99	660
LANGUAGE COWBOY	0.99	660
LAWRENCE LOVE	0.99	660
LEGEND JEDI	0.99	660
LESSON CLEOPATRA	0.99	660

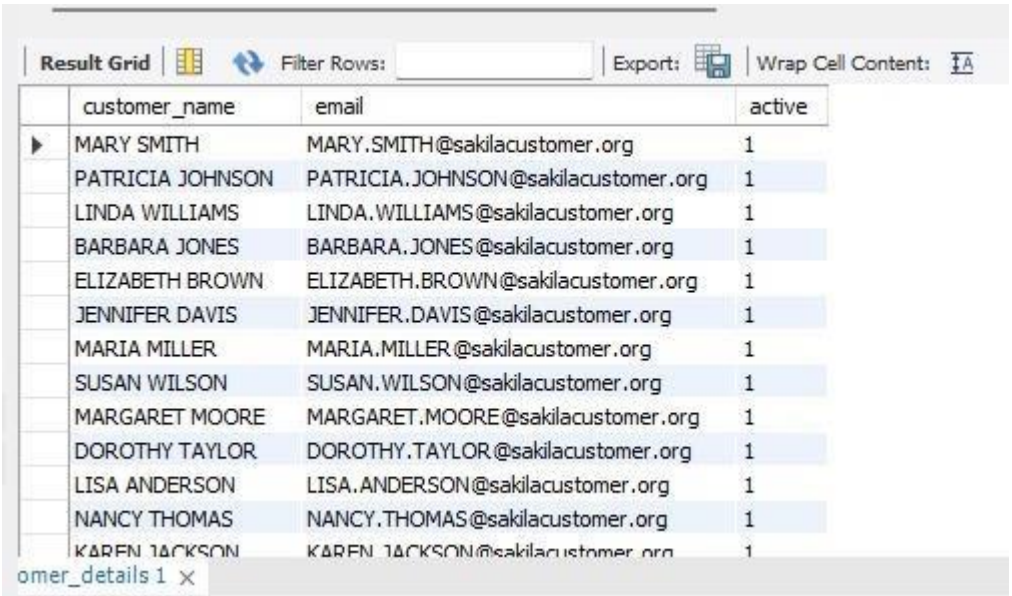
Result 2 ×

Q3. Create a view that displays customer names along with their email and active status.

Sol 3: ***Query:***

```
26 •  CREATE VIEW customer_details AS
27    SELECT
28        CONCAT(first_name, ' ', last_name) AS customer_name,
29        email,
30        active
31    FROM customer;
32
33    -- To view output:
34 •  SELECT * FROM customer_details LIMIT 5;
```

Output:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	email	active
MARY SMITH	MARY.SMITH@sakilacustomer.org	1
PATRICIA JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	1
LINDA WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	1
BARBARA JONES	BARBARA.JONES@sakilacustomer.org	1
ELIZABETH BROWN	ELIZABETH.BROWN@sakilacustomer.org	1
JENNIFER DAVIS	JENNIFER.DAVIS@sakilacustomer.org	1
MARIA MILLER	MARIA.MILLER@sakilacustomer.org	1
SUSAN WILSON	SUSAN.WILSON@sakilacustomer.org	1
MARGARET MOORE	MARGARET.MOORE@sakilacustomer.org	1
DOROTHY TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	1
LISA ANDERSON	LISA.ANDERSON@sakilacustomer.org	1
NANCY THOMAS	NANCY.THOMAS@sakilacustomer.org	1
KAREN JACKSON	KAREN.JACKSON@sakilacustomer.org	1

omer_details 1 ×

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

customer_name	email	active
MARION OCAMPO	MARION.OCAMPO@sakilacustomer.org	1
TRACY HERRMANN	TRACY.HERRMANN@sakilacustomer.org	1
SETH HANNON	SETH.HANNON@sakilacustomer.org	1
KENT ARSENAULT	KENT.ARSENAULT@sakilacustomer.org	1
TERRANCE ROUSH	TERRANCE.ROUSH@sakilacustomer.org	0
RENE MCALISTER	RENE.MCALISTER@sakilacustomer.org	1
EDUARDO HIATT	EDUARDO.HIATT@sakilacustomer.org	1
TERRENCE GUNDER...	TERRENCE.GUNDERSON@sakilacustome...	1
ENRIQUE FORSYTHE	ENRIQUE.FORSYTHE@sakilacustomer.org	1
FREDDIE DUGGAN	FREDDIE.DUGGAN@sakilacustomer.org	1
WADE DELVALLE	WADE.DELVALLE@sakilacustomer.org	1
AUSTIN CINTRON	AUSTIN.CINTRON@sakilacustomer.org	1

omer_details 1 ×

Q4. Find the top 10 customers who generated the highest payment amount.

Sol 4: ***Query:***

```
36 •  SELECT
37        c.customer_id,
38        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
39        SUM(p.amount) AS total_payment
40    FROM customer c
41    JOIN payment p ON c.customer_id = p.customer_id
42    GROUP BY c.customer_id, c.first_name, c.last_name
43    ORDER BY total_payment DESC
44    LIMIT 10;
```

Output:

Result Grid | Filter Rows: | Expo

customer_id	customer_name	total_payment
526	KARL SEAL	221.55
148	ELEANOR HUNT	216.54
144	CLARA SHAW	195.58
137	RHONDA KENNEDY	194.61
178	MARION SNYDER	194.61
459	TOMMY COLLAZO	186.62
469	WESLEY BULL	177.60
468	TIM CARY	175.61
236	MARCIA DEAN	175.58
181	ANA BRADLEY	174.66

Q5. Identify the customers whose total spending on rentals falls within the top 20% of all customers.

Sol 5: ***Query:***

```
46  WITH customer_spending AS (
47      SELECT
48          c.customer_id,
49          CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
50          SUM(p.amount) AS total_spent,
51          NTILE(5) OVER (ORDER BY SUM(p.amount) DESC) AS tile
52      FROM customer c
53      JOIN payment p ON c.customer_id = p.customer_id
54      GROUP BY c.customer_id, c.first_name, c.last_name
55  )
56  SELECT
57      customer_id,
58      customer_name,
59      total_spent
60  FROM customer_spending
61  WHERE tile = 1
62  ORDER BY total_spent DESC;
```

Output:

Result Grid | Filter Rows: | Export:

customer_id	customer_name	total_spent
526	KARL SEAL	221.55
148	ELEANOR HUNT	216.54
144	CLARA SHAW	195.58
137	RHONDA KENNEDY	194.61
178	MARION SNYDER	194.61
459	TOMMY COLLAZO	186.62
469	WESLEY BULL	177.60
468	TIM CARY	175.61
236	MARCIA DEAN	175.58
181	ANA BRADLEY	174.66
176	JUNE CARROLL	173.63
259	LENA JENSEN	170.67
50	DIANE COLLINS	169.65

Q6. Display every actor along with the number of movies they acted in and assign Dense Rank based on movie count.

Sol 6: ***Query:***

```
65 •  SELECT
66        a.actor_id,
67        CONCAT(a.first_name, ' ', a.last_name) AS actor_name,
68        COUNT(fa.film_id) AS movie_count,
69        DENSE_RANK() OVER (ORDER BY COUNT(fa.film_id) DESC) AS dense_rank_num
70    FROM actor a
71    JOIN film_actor fa ON a.actor_id = fa.actor_id
72    GROUP BY a.actor_id, a.first_name, a.last_name;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap

actor_id	actor_name	movie_count	dense_rank_num
107	GINA DEGENERES	42	1
102	WALTER TORN	41	2
198	MARY KEITEL	40	3
181	MATTHEW CARREY	39	4
23	SANDRA KILMER	37	5
81	SCARLETT DAMON	36	6
13	UMA WOOD	35	7
37	VAL BOLGER	35	7
60	HENRY BERRY	35	7
106	GROUCHO DUNST	35	7
144	ANGELA WITHERS...	35	7
158	VIVIEN BASINGER	35	7
65	ANGELA HUDSON	34	8
92	KIRSTEN AKROYD	34	8
105	SIDNEY CROWE	34	8
108	WARREN NOLTE	34	8

Result Grid | Filter Rows: | Export: | Wrap

actor_id	actor_name	movie_count	dense_rank_num
88	KENNETH PESCI	20	22
91	CHRISTOPHER BE...	20	22
135	RITA REYNOLDS	20	22
153	MINNIE KILMER	20	22
160	CHRIS DEPP	20	22
200	THORA TEMPLE	20	22
1	PENELOPE GUINESS	19	23
30	SANDRA PECK	19	23
63	CAMERON WRAY	19	23
183	RUSSELL CLOSE	19	23
31	SISSY SOBIESKI	18	24
71	ADAM GRANT	18	24
186	JULIA ZELLWEGER	16	25
35	JUDY DEAN	15	26
199	JULIA FAWCETT	15	26
148	EMILY DEE	14	27

22

Q7. Create a view showing movie title, category, rental rate and replacement cost.

Sol 7: *Query*:

```
75 •  CREATE VIEW movie_pricing_info AS
76    SELECT
77        f.title,
78        c.name AS category,
79        f.rental_rate,
80        f.replacement_cost
81    FROM film f
82    JOIN film_category fc ON f.film_id = fc.film_id
83    JOIN category c ON fc.category_id = c.category_id;
84
85    -- To view output:
86 •  SELECT * FROM movie_pricing_info LIMIT 5;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

title	category	rental_rate	replacement_cost
AMADEUS HOLY	Action	0.99	20.99
AMERICAN CIRCUS	Action	4.99	17.99
ANTITRUST TOMATOES	Action	2.99	11.99
ARK RIDGEMONT	Action	0.99	25.99
BAREFOOT MANCHURIAN	Action	2.99	15.99

Q8. Create a stored procedure that returns the top 20 most rented movies

Sol 8: ***Query:***

```
90  CREATE PROCEDURE GetTop20RentedMovies()
91  BEGIN
92      SELECT
93          f.film_id,
94          f.title,
95          COUNT(r.rental_id) AS rental_count
96      FROM film f
97      JOIN inventory i ON f.film_id = i.film_id
98      JOIN rental r ON i.inventory_id = r.inventory_id
99      GROUP BY f.film_id, f.title
100     ORDER BY rental_count DESC
101     LIMIT 20;
102 END //
103 DELIMITER ;
104 -- Execute procedure:
105 CALL GetTop20RentedMovies();
```

Output:

Result Grid | Filter Rows: | Export:

film_id	title	rental_count
103	BUCKET BROTHERHOOD	34
738	ROCKETEER MOTHER	33
730	RIDGEMONT SUBMARINE	32
382	GRIT CLOCKWORK	32
331	FORWARD TEMPLE	32
767	SCALAWAG DUCK	32
489	JUGGLER HARDLY	32
1000	ZORRO ARK	31
753	RUSH GOODFELLAS	31
369	GOODFELLAS SALUTE	31
31	APACHE DIVINE	31
735	ROBBERS JOON	31
621	NETWORK PEAK	31
418	HOBBIT ALIEN	31
891	TIMBERLAND SKY	31
973	WIFE TURN	31
702	PULP BEVERLY	30
609	MUSCLE BRIGHT	30
403	HARRY IDAHO	30
285	ENGLISH BULWORTH	30

Q9. Create a procedure that accepts a movie rating and returns all movies of that rating.

Sol 9: ***Query:***

```
112 •  CREATE PROCEDURE GetMoviesByRating(IN p_rating VARCHAR(10))
113    BEGIN
114        SELECT
115            film_id,
116            title,
117            rating,
118            rental_rate,
119            replacement_cost
120        FROM film
121        WHERE rating = p_rating;
122    END //
123
124    DELIMITER ;
125
126 •  -- Execute procedure:
127    CALL GetMoviesByRating('PG-13');
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

film_id	title	rating	rental_rate	replacement_cost
7	AIRPLANE SIERRA	PG-13	4.99	28.99
9	ALABAMA DEVIL	PG-13	2.99	21.99
18	ALTER VICTORY	PG-13	0.99	27.99
28	ANTHEM LUKE	PG-13	4.99	16.99
33	APOLLO TEEN	PG-13	2.99	15.99
35	ARACHNOPHOBIA ROLLERCOASTER	PG-13	2.99	24.99
36	ARGONAUTS TOWN	PG-13	0.99	12.99
44	ATTACKS HATE	PG-13	4.99	21.99
45	ATTRACTION NEWTON	PG-13	4.99	14.99
48	BACKLASH UNDEFEATED	PG-13	4.99	24.99
57	BASIC EASY	PG-13	2.99	18.99
64	BEETHOVEN EXORCIST	PG-13	0.99	26.99
67	BERETS AGENT	PG-13	2.99	24.99
71	BILKO ANONYMOUS	PG-13	4.99	25.99
73	BINGO TALENTED	PG-13	2.99	22.99
79	BLADE POLISH	PG-13	0.99	10.99
81	BLINDNESS GUN	PG-13	4.99	29.99
94	BRAVEHEART HUMAN	PG-13	2.99	14.99
96	BREAKING HOME	PG-13	2.99	21.99
98	BRIGHT ENCOUNTERS	PG-13	4.99	12.99

Q10. Identify the top 3 films in each category based on their rental counts.

Sol 10: ***Query:***

```
WITH film_ranks AS (
    SELECT
        c.name AS category_name,
        f.title,
        COUNT(r.rental_id) AS rental_count,
        DENSE_RANK() OVER (
            PARTITION BY c.category_id
            ORDER BY COUNT(r.rental_id) DESC
        ) AS rank_num
    FROM category c
    JOIN film_category fc ON c.category_id = fc.category_id
    JOIN film f ON fc.film_id = f.film_id
    JOIN inventory i ON f.film_id = i.film_id
    JOIN rental r ON i.inventory_id = r.inventory_id
    GROUP BY c.category_id, c.name, f.film_id, f.title
    GROUP BY c.category_id, c.name, f.film_id, f.title
)
SELECT
    category_name,
    title,
    rental_count,
    rank_num
FROM film_ranks
WHERE rank_num <= 3
ORDER BY category_name, rank_num;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	title	rental_count	rank_num
Action	RUGRATS SHAKESPEARE	30	1
Action	SUSPECTS QUILLS	30	1
Action	HANDICAP BOONDOCK	28	2
Action	STORY SIDE	28	2
Action	TRIP NEWTON	28	2
Action	PRIMARY GLASS	27	3
Animation	JUGGLER HARDLY	32	1
Animation	DOGMA FAMILY	30	2
Animation	STORM HAPPINESS	29	3
Children	ROBBERS JOON	31	1
Children	IDOLS SNATCHERS	30	2
Children	SWEETHEARTS SUSPECTS	29	3
Classics	TIMBERLAND SKY	31	1
Classics	FROST HEAD	30	2
Classics	GILMORE BOILED	28	3
Classics	VOYAGE LEGALLY	28	3
Comedy	ZORRO ARK	31	1
Comedy	CAT CONEHEADS	30	2
Comedy	CLOSER BANG	28	3
Documentary	WIFE TURN	31	1

Q11. Calculate the running total of rentals per category, ordered by rental count.

Sol 11: ***Query:***

```
155  WITH category_rentals AS (
156      SELECT
157          c.name AS category_name,
158          COUNT(r.rental_id) AS rental_count
159      FROM category c
160      JOIN film_category fc ON c.category_id = fc.category_id
161      JOIN film f ON fc.film_id = f.film_id
162      JOIN inventory i ON f.film_id = i.film_id
163      JOIN rental r ON i.inventory_id = r.inventory_id
164      GROUP BY c.category_id, c.name
165  )
166  SELECT
167      category_name,
168      rental_count,
169      SUM(rental_count) OVER (ORDER BY rental_count ASC) AS running_total
170  FROM category_rentals;
171
```

Output:

Result Grid | Filter Rows: | Export:

category_name	rental_count	running_total
Music	830	830
Travel	837	1667
Horror	846	2513
Classics	939	3452
New	940	4392
Comedy	941	5333
Children	945	6278
Games	969	7247
Foreign	1033	8280
Documentary	1050	9330
Drama	1060	10390
Family	1096	11486
Sci-Fi	1101	12587
Action	1112	13699
Animation	1166	14865
Sports	1179	16044

Q12. Create a CTE to generate a report showing pairs of actors who have appeared in the same film together, using the film_actor table

Sol 12: ***Query:***

```
WITH actor_pairs AS (
    SELECT
        fa1.film_id,
        CONCAT(a1.first_name, ' ', a1.last_name) AS actor_1,
        CONCAT(a2.first_name, ' ', a2.last_name) AS actor_2
    FROM film_actor fa1
    JOIN film_actor fa2
        ON fa1.film_id = fa2.film_id
        AND fa1.actor_id < fa2.actor_id
    JOIN actor a1 ON fa1.actor_id = a1.actor_id
    JOIN actor a2 ON fa2.actor_id = a2.actor_id
)
SELECT
    actor_1,
    actor_2,
    COUNT(film_id) AS co_starred_count
FROM actor_pairs
GROUP BY actor_1, actor_2
ORDER BY co_starred_count DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap C

actor_1	actor_2	co_starred_count
JULIA MCQUEEN	HENRY BERRY	7
CUBA OLIVIER	MARY KEITEL	6
KIRSTEN PALTROW	WARREN NOLTE	6
VAL BOLGER	ROCK DUKAKIS	6
BEN WILLIS	HARVEY HOPE	6
MORGAN MCDORMAND	WILL WILSON	6
MENA TEMPLE	JAYNE NOLTE	5
HENRY BERRY	LIZA BERGMAN	5
ANGELA HUDSON	CUBA ALLEN	5
ANGELA HUDSON	ALAN DREYFUSS	5
ANGELINA ASTAIRE	SUSAN DAVIS	5
BEN WILLIS	SUSAN DAVIS	5
SEAN GUINESS	MARY KEITEL	5
ELLEN PRESLEY	CAMERON ZELL...	5
DARYL WAHLBERG	MATTHEW CAR...	5
SUSAN DAVIS	ROCK DUKAKIS	5
SUSAN DAVIS	SCARLETT BENI...	5
WALTER TORN	EWAN GOODING	5
SYLVESTER DERN	JEFF SILVERST...	5
JOHNNY LOLLOBRIGIDA	SUSAN DAVIS	5

Result 11

Q13. Create a procedure that accepts a customer ID as input and returns the customer's total payment as an output parameter.

Sol 13: ***Query:***

```
196  CREATE PROCEDURE GetCustomerTotalPayment(
197      IN p_customer_id INT,
198      OUT p_total_payment DECIMAL(10,2)
199  )
200  BEGIN
201      SELECT COALESCE(SUM(amount), 0.00)
202      INTO p_total_payment
203      FROM payment
204      WHERE customer_id = p_customer_id;
205  END //
206
207  DELIMITER ;
208
209  -- Execute Procedure for Customer ID = 1:
210  CALL GetCustomerTotalPayment(1, @total_paid);
211  SELECT @total_paid AS customer_total_payment;
```

Output:

